

Frontend Interview Prep Assignment (React + TypeScript)

Overview

Build 5 React + TypeScript features in one GitHub repository. Ship each one as its own pull request, then record one 2-minute video explaining all five. You may use AI tools, but in the interview you must be able to explain the code you submit.

- **Stack:** React and TypeScript (strict mode). Choose your own tooling and libraries: build tool, routing, forms, state, styling and testing.
- **Time limit:** 2 hours for all 5 questions, including PRs and merges. Suggested split: Q1 and Q2 15 minutes each, Q3 and Q4 25 minutes each, Q5 40 minutes. You get 30 more minutes after the cut-off to record and upload the video.
- **What we assess:** working UI, clean and typed components, tests, Git/PR discipline, and above all whether you understand what you built.
- **Rule:** don't use a UI or table library that does the core work of a question for you.

Claude Code is expected. Use it to write the code fast, and spend the time you save understanding what it wrote. The questions describe *what* to build, not *how*; the design decisions are yours. In the interview, be ready to explain every decision and the alternatives you considered, and to make a short live change.

Submission workflow

Each question is one branch, one PR and one merge, in order from Q1 to Q5.

1. Create a public GitHub repo named `fe-interview-prep`. On `main`, set up a React + TypeScript app with a linter and a test runner. Each question lives on its own page or route.
2. For each question, create a branch from the latest `main`: `feature/q1-todo`, `feature/q2-search`, `feature/q3-form-wizard`, `feature/q4-data-table`, `feature/q5-auth-flow`.
3. Commit in small, meaningful steps, such as `Add search input` or `Ignore outdated search results`. Avoid commits like `fix`, `changes` or `final`.
4. Open a PR into `main` using the template below. Add screenshots or a GIF of the UI. Review your own diff before merging.
5. Merge the PR, then pull `main` before starting the next branch.
6. After the last merge, record the video, upload it to YouTube and add the link to the README.

PR description template

Problem

What this PR solves (1–2 lines).

Approach

How the components and state are organised.

Decisions & trade-offs

What you chose, and why over the alternatives.

Screenshots

UI before/after or a GIF.

How to test

Page to open, steps, test files.

README (keep it updated as you merge; add the video link at the end):

#	Question	PR link
1	Todo App	
2	Live Search	
3	Registration Wizard	
4	Data Table	
5	Login & Session Handling	

Video:

Video guidelines

After the last merge, record **one screen recording of at most 2 minutes** covering all 5 questions. Upload it to YouTube as **Unlisted** and add the link to the README.

Time	What to show
0:00–0:10	Quick tour of the repo and the merged PRs
0:10–1:50	About 20 seconds per question: what it does, and the most important decision you made
1:50–2:00	One thing you would improve with more time

- With 20 seconds per question, skip the line-by-line walkthrough. Show the one piece of code that matters most and say *why* you built it that way.
- Use any recorder: Loom, OBS, QuickTime or the built-in screen recorder.
- Keep your face camera optional and your voice clear. Zoom the editor font to at least 16px.

AI usage policy

AI tools (Claude, Copilot, ChatGPT) are allowed. Not understanding your own code is not. The interviewer will open your PRs and ask follow-up questions like these:

- Why does this component re-render? Could you prevent it?
- Why is this value stored rather than calculated?
- What happens if the user types fast, or leaves the page mid-request?
- Why did you choose this library or approach? What alternatives did you consider?
- Change this behaviour live.

If you can't answer these about your own code, the question counts as not done.

The questions

Q1 — Todo App

Build a todo app that survives a page refresh.

Requirements

- Add, edit, delete and complete todos.
- Filter by All / Active / Completed. Show how many items are left, and offer a "Clear completed" button.
- The todos and the selected filter survive a page refresh.
- Ignore empty or whitespace-only titles.
- Write the saving logic so other features could reuse it.

Acceptance criteria

- Refreshing the page keeps the todos and the filter.
- At least one automated test.

Q2 — Live Search

Build a search box that shows results from a public API as the user types.

Requirements

- Use a public API, for example `https://dummyjson.com/products/search?q=` or GitHub user search.
- Don't send a request on every keystroke. Search only once the user briefly stops typing.
- The results on screen always match the latest query, even if an earlier response arrives late.
- Show loading, error (with retry), empty ("No results for 'xyz'") and results states.
- Highlight the matched text in each result.

Acceptance criteria

- Typing "react" quickly sends a single request (visible in the Network tab).
- At least one automated test.

Q3 — Registration Wizard

Build a 3-step registration form with validation and a review step.

Requirements

- Step 1 is personal info (name, email, phone). Step 2 is address (country, city, postal code). Step 3 is preferences (a plan, and a list of skills the user can add or remove, with at least one required).
- The user can't move to the next step until the current one is valid. Going back keeps what they entered.
- For India, the postal code must be 6 digits. For other countries, any postal code is accepted.
- A review step shows everything, with a way to edit each step. Submitting simulates an API call and shows a success message.
- Show a progress indicator.
- Refreshing the page mid-way keeps the user's progress.

Acceptance criteria

- An invalid step blocks moving forward and shows clear errors.
- At least one automated test.

Q4 — Data Table

Build a reusable table component and use it to browse a large dataset.

Requirements

- The table can be reused for any kind of data and is configured by describing its columns.
- Show 500+ rows, for example from <https://dummyjson.com/users?limit=0>.
- Clicking a column header sorts by it, cycling ascending → descending → none. Add a global search and at least one column filter.
- Paginate, with a page-size choice of 10, 25 or 50.
- The current view (sort, search, filters and page) can be shared as a link and works with the browser's back/forward buttons.
- Don't use a table library.

Acceptance criteria

- Opening a shared link restores the exact view.
- Changing a filter goes back to page 1.
- At least one automated test.

Q5 — Login & Session Handling

Build a login flow where users stay logged in smoothly while short-lived tokens expire in the background.

Requirements

- Mock a backend (any approach) with endpoints for login, token refresh, current user, orders and admin stats. The access token expires after 30 seconds and the refresh token after 10 minutes.

- Users can log in and out. Protected pages send logged-out users to login, then back to the page they wanted.
- One page is visible only to admins.
- When the access token expires, the user shouldn't notice: failed requests recover automatically.
- If several requests fail at once because of expiry, the app makes only one refresh call.
- If the refresh fails, log the user out and go to the login page.
- After a page reload, restore the session without flashing the login page.
- Decide where to keep the tokens, and be ready to justify it.

Acceptance criteria

- When 3 requests fire in parallel after expiry, the Network tab shows exactly one refresh call.
- At least one automated test proving this.

If you finish early (optional)

None of these are required. Finish all 5 core questions first.

- Q1: reorder todos by dragging; keep the filter in the URL.
- Q2: full keyboard navigation and screen-reader support; reuse results for repeated queries.
- Q3: accessible error messages; move focus to the first error.
- Q4: a server-side mode where each page is fetched; a sticky header; show/hide columns.
- Q5: logging out in one tab logs out the other tabs; an idle-timeout warning.

Submission checklist

- ☐ Public repo `fe-interview-prep` created, with a README
- ☐ 5 branches, 5 PRs, all merged into `main`
- ☐ Every PR uses the description template and includes screenshots or a GIF
- ☐ The README explains how to run the app and the tests
- ☐ Tests, lint and type-check all pass
- ☐ Code and PRs done within 2 hours; one video of at most 2 minutes uploaded to YouTube (Unlisted) within 30 minutes after
- ☐ The README has every PR link and the YouTube link
- ☐ You can explain every file in the repo without AI help